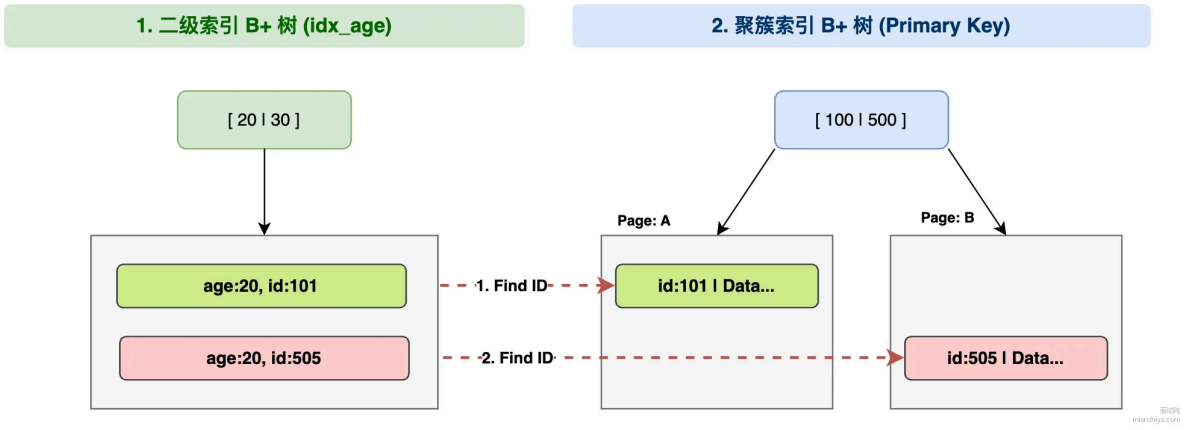
MySQL 中的回表是什么？ - 面试鸭 | 2026最新面试题+详细答案解析

> ## Excerpt
>
> 2026最新后端、MySQL、数据库面试题，包含详细答案解析和练习题，29436+程序员已学习，已帮助10万+程序员通过校招/社招/实习面试，拿到offer！

回答重点

回表是指用二级索引查询时，因为二级索引的叶子节点只存索引列的值和主键值，拿不到其他字段的数据。如果查询需要其他字段，就得拿着主键再去聚簇索引查一遍完整的数据行，这个过程就叫回表。

简单说，二级索引不够用，得回到主键索引再查一次。

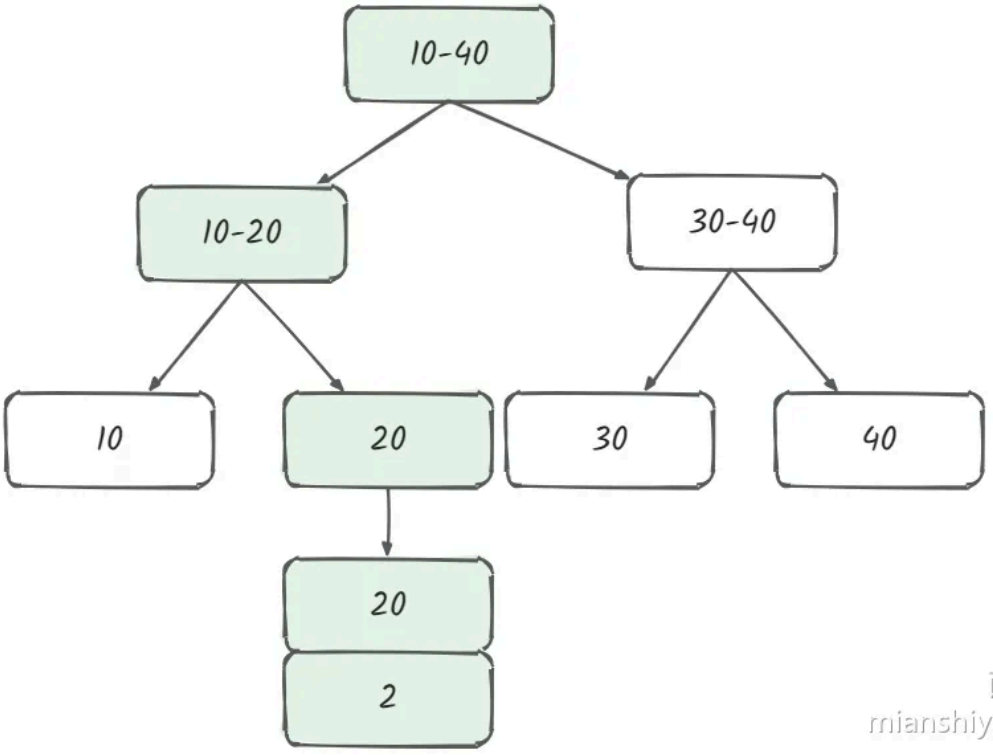


扩展知识

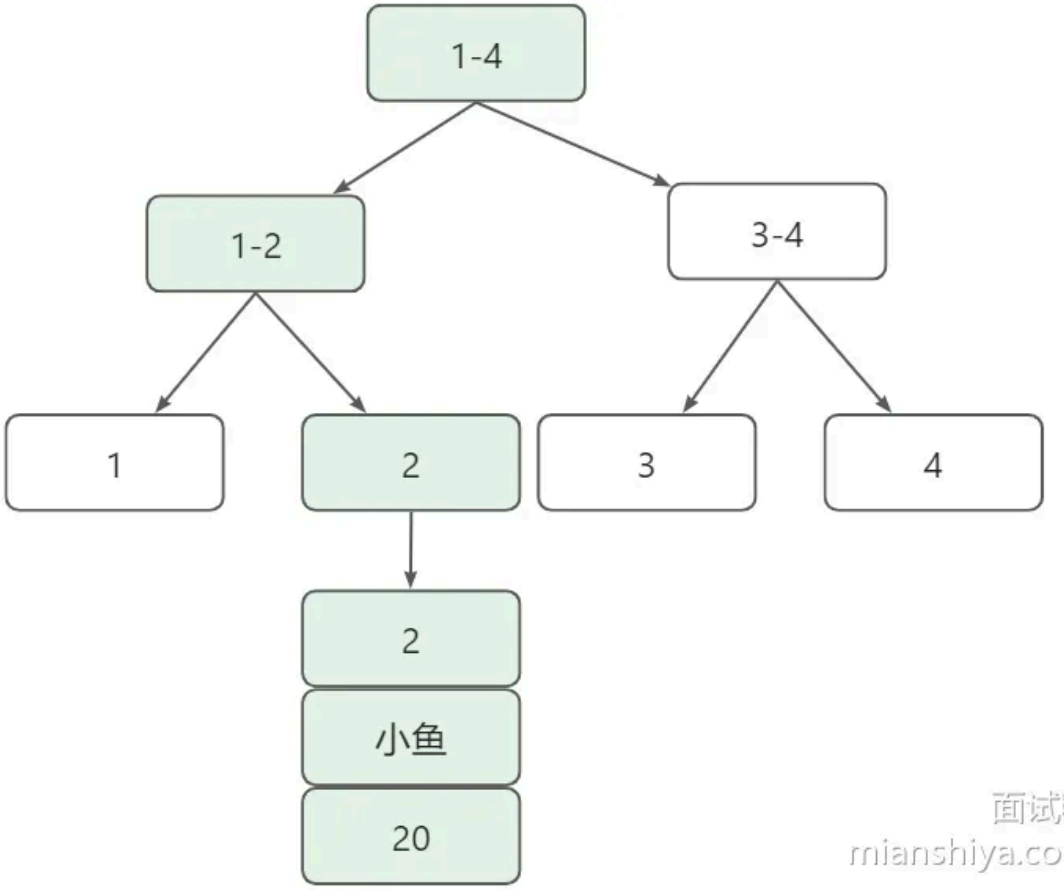
回表的完整流程

举个例子：`select * from user where age = 20;`，user 表有 id、name、age 三个字段，age 上有索引。

1）先走 age 二级索引，找到 age = 20 的记录，拿到主键值：



2）因为用了 `select *`，需要 name、age、id 三个字段，但 age 索引里没有 name，得拿着主键去聚簇索引查完整数据：



最终拿到 id=2, name='小鱼', age=20 这条完整记录。

回表的性能代价

回表不只是多查一次那么简单，**还会带来大量随机 I/O。**

因为通过二级索引拿到的主键 id 不是连续的。比如 age = 20 的人很多，在 age 索引里按年龄排序，但对应的 id 是无序的，可能是 2、58、199、1024 这种散落分布。拿这些 id 去聚簇索引查，每次都要定位到不同的数据页，产生大量随机 I/O。

顺序 I/O 可以利用磁盘预读，一次读一大块连续数据，非常快。随机 I/O 每次都要重新寻址，磁盘得来回转，慢得多。所以如果查询返回几万行数据，每行都要回表，性能会急剧下降。

如何避免/减少回表

1）**覆盖索引**

如果查询需要的列都在索引里，就不用回表了。比如索引是 `(age, name)`，查询 `select name from user where age = 20`，索引叶子节点存了 age、name、主键，直接返回就行。

EXPLAIN 的 Extra 列显示 `Using index` 就说明走了覆盖索引，没有回表。

2）**索引下推**

MySQL 5.6 引入的优化。假设索引是 `(age, name)`，查询 `where age = 20 and name like '%张'`，以前的做法是先通过 age = 20 拿到所有主键，全部回表拿完整数据，再在 Server 层用 name 过滤。

有了索引下推，在引擎层就能用 name like '%张' 先过滤一遍，不符合条件的直接跳过不回表，大大减少回表次数。

3）**减少 select * 的使用**

不要图方便什么都用 `select *`。如果只需要索引里的字段，显式指定列名，能避免不必要的回表。

什么时候 MySQL 会放弃索引直接全表扫描

如果回表次数太多，MySQL 优化器可能认为还不如直接全表扫描划算。

经验值大概是：如果查询返回的数据量超过表的 15%-30%，优化器可能放弃二级索引。因为与其回表几十万次产生随机 I/O，不如全表顺序扫描一遍，虽然扫的数据多，但都是顺序 I/O，整体可能更快。

这也是为什么有时候明明建了索引，EXPLAIN 却显示全表扫描。不是索引没用，是优化器算了一笔账，觉得不走索引更快。

面试官追问

提问：一个查询走了索引但还是很慢，可能是什么原因？

> 详细解答可以看：MySQL 中使用索引一定有效吗？如何排查索引效果？

回答：大概率是回表次数太多。比如查询返回几十万行，每行都要回表拿数据，产生大量随机 I/O。可以看 EXPLAIN 的 rows 估算值，如果很大说明回表代价高。解决办法是用覆盖索引减少回表，或者优化查询条件缩小结果集。

提问：怎么判断一个查询有没有回表？

回答：看 EXPLAIN 的 Extra 列。如果显示 `Using index`，说明走了覆盖索引，没有回表。如果没有这个提示，或者显示 `Using index condition`，说明有回表。另外如果 type 是 index，表示全索引扫描，也可能有回表。

提问：主键查询需要回表吗？

回答：不需要。主键索引就是聚簇索引，叶子节点直接存完整数据行，一次查找就能拿到所有数据。回表只发生在二级索引查询时。